

Práctica Integradora

Sistema de Préstamo de Equipos Tecnológicos

Fecha:	23/06/2026
Duración:	3 horas
Modalidad:	Individual
Tecnologías:	Python, SQLAlchemy, MySQL, MVC y arquitectura por capas

Objetivo

Desarrollar un sistema utilizando Python, SQLAlchemy y MySQL para administrar equipos tecnológicos, estudiantes, categorías y préstamos, aplicando separación de responsabilidades mediante las capas config, entity, repository, service, controller y main.py.

Requerimientos

Una universidad presta equipos tecnológicos como laptops, proyectores, tablets y calculadoras a sus estudiantes. Actualmente el control se realiza de forma manual, provocando registros duplicados, dificultad para localizar equipos, préstamos sin seguimiento y errores al verificar disponibilidad o fechas de devolución.

El sistema debe permitir registrar la información principal, consultar datos, actualizar registros, eliminar cuando sea permitido y generar reportes útiles para la administración del préstamo de equipos.

Base de Datos

- Tabla estudiantes: id, nombre, correo, carrera.
- Tabla categorías: id, nombre.
- Tabla equipos: código, nombre, marca, estado, categoria_id.
- Tabla prestamos: id, estudiante_id, equipo_codigo, fecha_prestamo, fecha_devolucion, estado.

Reglas de Negocio

1. El código del equipo no puede repetirse.
2. Todo equipo debe pertenecer a una categoría existente.
3. Todo préstamo debe pertenecer a un estudiante existente.
4. No se puede prestar un equipo que no esté disponible.
5. La fecha de devolución debe ser válida.
6. No se puede eliminar un equipo con préstamos activos.

Funcionalidades

Gestión de estudiantes	Registrar, consultar, buscar, actualizar y eliminar estudiantes.
Gestión de categorías	Registrar, consultar, buscar, actualizar y eliminar categorías.
Gestión de equipos	Registrar, consultar, buscar, actualizar y eliminar equipos.
Gestión de préstamos	Registrar préstamos, consultar préstamos, buscar préstamos y devolver equipos.

Reportes

1. Mostrar todos los equipos ordenados por nombre.
2. Mostrar equipos por categoría indicada por el usuario.
3. Mostrar préstamos por estudiante.
4. Mostrar equipos disponibles.
5. Mostrar préstamos activos.

Estructura del Proyecto

```
config/  
controller/  
entity/  
repository/  
service/  
main.py
```

Script SQL Inicial

```
CREATE DATABASE prestamos_equipos_db;  
USE prestamos_equipos_db;  
  
CREATE TABLE estudiantes(  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  correo VARCHAR(100),  
  carrera VARCHAR(100)  
);  
  
CREATE TABLE categorias(  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE equipos(  
  codigo VARCHAR(20) PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  marca VARCHAR(50),  
  estado VARCHAR(30),  
  categoria_id INT,  
  FOREIGN KEY (categoria_id) REFERENCES categorias(id)  
);  
  
CREATE TABLE prestamos(  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  estudiante_id INT,  
  equipo_codigo VARCHAR(20),  
  fecha_prestamo DATE,  
  fecha_devolucion DATE,  
  estado VARCHAR(30),  
  FOREIGN KEY (estudiante_id) REFERENCES estudiantes(id),  
  FOREIGN KEY (equipo_codigo) REFERENCES equipos(codigo)  
);
```

Indicaciones de Desarrollo

El estudiante debe implementar el sistema respetando la estructura solicitada. Las entidades deben mapear correctamente las tablas de la base de datos. Los repositorios deben contener las operaciones de base de datos mediante ORM. Los servicios deben contener las validaciones y reglas de negocio. Los controladores deben gestionar la interacción con el usuario o con la vista.

No se debe concentrar toda la lógica en un solo archivo. Cada capa debe tener una responsabilidad clara y comunicarse con la capa correspondiente.

Criterios de Evaluación

Criterio	Puntos
Implementación correcta de la conexión con MySQL y SQLAlchemy	10
Implementación correcta del mapeo de tablas al lenguaje mediante entidades ORM	20
Define correctamente las acciones en base de datos mediante repository y ORM	20
Realiza las validaciones indicadas en reglas de negocio dentro de service	20
Gestiona correctamente la interacción de main/controller con las otras capas	10
Muestra los reportes correctamente	10
Claridad del código y organización del proyecto	10
Total	100

Resultado Esperado

Al finalizar, el sistema debe permitir administrar estudiantes, categorías, equipos y préstamos, aplicar las reglas de negocio, evitar registros inválidos y mostrar los reportes solicitados. La solución debe evidenciar el uso correcto de ORM, arquitectura por capas y organización modular.